

Titan Vision // Sentry-Core

Computer Vision & Spatial Intelligence Master Guide

Syed Mutahir Hussain

Final Year Computer Science Student | UET Taxila

Expertise: AI, Data Science, and Machine Learning

1. Project Vision

Titan Vision is a real-time security monitoring framework. Unlike standard "object detectors" that simply draw boxes around people, Titan Vision implements **Spatial Logic**. It uses geofencing to determine if a person is in a "Restricted Zone," triggers automated evidence logging, and provides a professional-grade HUD (Heads-Up Display) for human operators.

2. The Detection Engine: YOLOv8

We utilized the **YOLOv8 (You Only Look Once)** architecture by Ultralytics.

Why YOLOv8 vs. Faster R-CNN or SSD?

Faster R-CNN: While highly accurate, it is a "two-stage" detector. It first proposes regions and then classifies them. This is too slow for real-time CPU use.

SSD (Single Shot Detector): Fast, but often struggles with small objects or overlapping targets.

YOLOv8: It is a "one-stage" detector that views the entire image as a single regression problem. It is significantly more efficient. Specifically, we used the **yolov8n.pt** (Nano) model, which is optimized for **Edge AI** and CPU-only devices like your Ryzen 5.

3. Spatial Intelligence: The Geofence

The most important engineering feat in this project is the **Mathematical Geofence**.

The Centroid Logic

Instead of checking if any part of the bounding box touches the restricted zone (which causes false alarms), we calculate the **Centroid** (the exact center point) of the detected person using the formula:

$$Cx = (x1 + x2) / 2 \quad | \quad Cy = (y1 + y2) / 2$$

Point-in-Polygon Testing

We used OpenCV's `cv2.pointPolygonTest()`. This is a vector math function that determines if a point is inside, outside, or on the boundary of a complex polygon. This allows us to define any area of a room as "restricted" regardless of its shape.

4. Software Architecture: The Sentry-Core HUD

We built the UI using **CustomTkinter** and an **Event-Driven Design**.

The Challenge: Threading and UI Freezing

Standard Python loops (`while True`) freeze Graphical User Interfaces because they block the main thread. To fix this, we used the `.after()` method. This allows the UI to process a frame, "check" for button clicks, and then schedule the next frame 10ms later. This keeps the application responsive while running heavy AI inference.

Key Features of the HUD:

- **Dynamic Thresholding:** A real-time slider that adjusts the Confidence level of the AI without restarting the script.
- **Automated Auditing:** A cooling-down file-save system that captures intruders but prevents the hard drive from being flooded with thousands of identical photos.
- **Terminal Integration:** A custom text box that mimics a Linux terminal for live event logging.

5. Interview Cheat Sheet: The "Hard" Questions

Q: Why use Python for real-time vision if C++ is faster?

A: While C++ is the standard for production edge-deployment, Python allows for rapid prototyping and integration with modern AI libraries like Ultralytics. To bridge the speed gap, I used **NumPy** for vectorized math and **Stream-based inference** to minimize latency.

Q: How would you handle false positives (e.g., a chair being called a person)?

A: In this project, I implemented a **Confidence Threshold** (default 0.6). By requiring

the model to be at least 60% sure, we filter out noise. For a production system, I would add **Temporal Persistence**—requiring the person to be detected for 5 consecutive frames before triggering an alarm.

Q: How do you optimize for your CPU hardware?

A: I selected the **Nano** architecture for YOLOv8. I also used **verbose=False** and disabled the default library plotting to save CPU cycles for the actual logic and UI rendering.